

BigData Assignment 2 Report

Artem Levakov
a.levakov@innopolis.university

Abstract—This assignment compares three major database management systems: PostgreSQL (relational), MongoDB (document-oriented), and Neo4j (graph), across identical analytical workloads. The goal is to evaluate performance, scalability, and suitability for different query patterns. Report contains information about design, implementation and benchmarking of these databases. Dataset used for testing contained information about people, products, and friendships among people, as well as product relationships and marketing campaign interactions. As a result of the experiments, there is no single model that is superior to the others as all three databases have their advantages and disadvantages. PostgreSQL performed best for analytical queries (Campaign Analysis: 7.72s vs 26.19s vs 101.78s; Recommendations: 0.026s vs 0.034s vs 2.24s), while Neo4j excelled at graph traversal queries (Full-Text Search: 2.29s vs 3.39s vs 16.19s). Therefore, the database selection should be decided based on the application domain and query patterns.

I. HARDWARE AND SOFTWARE

TABLE I
HARDWARE SPECIFICATIONS

Component	Specifications
OS	Windows 10 PRO version 22H2 (build 19045)
CPU	AMD Ryzen 5 5600H
CPU clock	3.3 GHz (6 cores / 8 threads)
RAM	16 GB
Platform	HONOR MagicBook 16

TABLE II

Software	Version	Deployment
PostgreSQL	16.0	Local (Windows)
MongoDB	v8.2.5	Local (Windows)
Neo4j	2026.1.4 (enterprise)	Local (Windows)

II. DATA MODELING

A. Dataset overview

The source dataset is 5 CSV files, a part of an export of an e-commerce platform, containing information about:

- **Clients:** registered users who made a purchase. User who didn't made a purchase don't show up in this file.
- **Friendships:** directed social graph connections between users, representing mutual friendships.
- **Events:** user interaction records - view, cart, purchase - each linked to a product, a category, and optionally a brand and price
- **Campaigns:** data of each marketing campaign
- **Messages:** marketing campaign messages sent to clients, with flags such as: is_opened, is_clicked, is_purchased.

B. Data EDA

After performing small analysis of data, the data was cleaned by clean_data.py script - string 't/f' was converted to True/False, some of the datetime values were processed. The full empty columns were dropped. One of the problems is table Campaigns - it had some duplicating ids, so it was fixed using surrogate id when uploading to the databases. Big portion of data contained null values, because of that, it affected the modeling of databases.

C. PostgreSQL Model

Note before all schemes - they may not be exact copy of real databases structure due to Hackolade limitations. To check real structure it is recommended to check code for data loading in databases.

The postgresql schema was based on 3NF rule, but not fully. Parts of database were denormalized for better performance in queries. This denormalization won't affect in long-term, so I assumed that it is okay to do it. Some parts of original tables was divided in subtables, for example, engagement part of messages - timestamps of bool flags were moved to subtables. The schema of PSQL was made in Hackolade and is shown below:

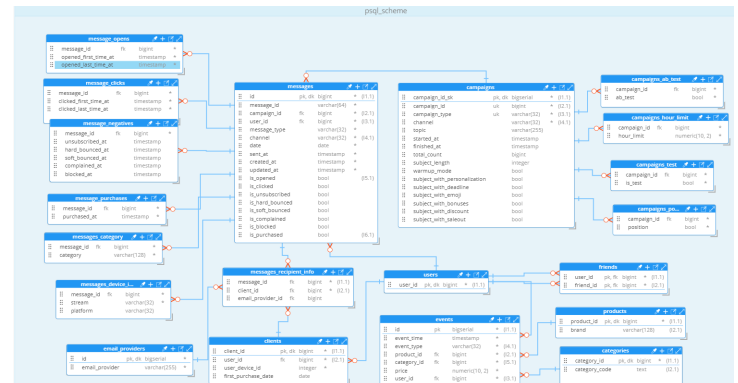


Fig. 1. Screenshot of PostgreSQL Hackolade schema

D. MongoDB Model

MongoDB stores each entity type as a separate collection in json format. In MongoDB we don't have 'join' operators, because foreign keys doesn't exist in this database (On the scheme you can see keys, but in real database they do not exist). Instead we have 'lookup' operator - it takes much more time to do it compare to regular 'join'. That's why in mongo we make embedded documents, for example, in messages

collection we store not only information of message, but also all information about campaign that this message delivered. Using that we don't have to do 'lookup' to have information when we are working with messages. This will increase the speed of the query, but will take more space to store. It's one of the main features of MongoDB approaches to store data.

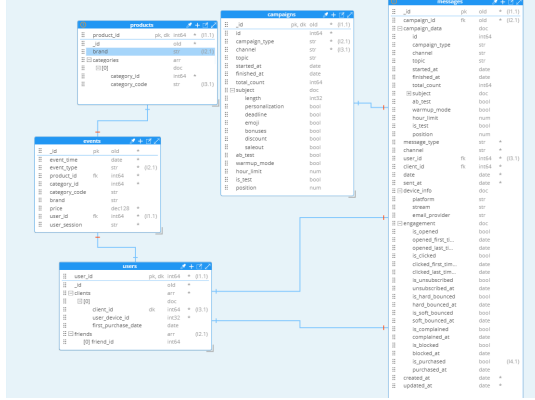


Fig. 2. Screenshot of MongoDB Hackolade schema

E. Neo4j Model

Neo4j is a graph database, each object is a node, that contain some properties in which we store data. Each node is connected with each other by relationships, which contain poperties too. Using graph approach, i embedded messages and events as relationships between nodes. You can see the schema for Neo4j below:

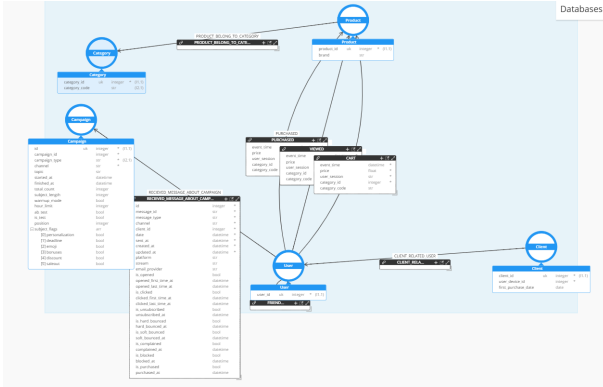


Fig. 3. Screenshot of Neo4j Hackolade schema

III. EXPERIMENTAL RESULTS

A. Benchmark Methodology

Each query script was executed 5 times per database. Queries was executed via python and timing was measured using python time library. All results were produced on the same machine, with all extra applications closed. PostgreSQL and Neo4j queries were executed via files by python, MongoDB queries were copied as pipeline in python and executed directly. This is because python works badly with .js code.

The code of all queries have the same logic. The code itself won't be listed here, you can check it in the repository of the project. The exact guide how to reproduce the experiment is written in README file on the project.

B. Query 1 - Campaign Analysis

The question was: "Analyze the data to find out whether campaigns attracted the customers to purchase products." The logic of the query for this task was: find users who received messages from campaigns, check if they purchased after receiving message, rank campaigns by total purchase conversion rate. Limit results by top 20 campaigns. The result of running this query for every database is the same. you can check them in output files in project.

C. Query 2 - Recommendations

The question was: "The business has logged behavior of customers on the platform. Use this information to find out which top personalized recommended products we can display in the home page of the user." I run this query exactly for 1 user. To have most interesting results, i firstly find the most active user by PostgreSQL query 'find_active_user', then using his id, i run all other queries. This id of most active user in my data was - 560126337.

The logic of the query for this task was: analyze given user's purchase history as highest weight, analyze given user's view/cart history as medium and lowest weights, find similar users via friends, rank products by interaction score and output 20 top results.

The result of running this query for every database is the same, except for postgre - this it because of attempt to normalize the data. Some categories of product have one id listed with category code and without. That's why they may lose the code when uploading to postgre.

D. Query 3 - Full-Text Search

The question was: "Based on the top products you obtained above, find out which products can be retrieved by full text search for products based on keywords from the category_code of the products."

Results of previous query were hardcoded for optimization in each implementation. The logic of the query for this task was: take previous results, extract unique category_codes from them, divide text by '.' into keywords, find products matching keywords, match quality of keywords and output 20 top results.

E. Performance Results

Table 3 reports average and standard deviation of execution times over 5 runs. Figure 4 is a bar plot of average execution time. Figure 5 is a bar plot of standard deviation of runs.

IV. DISCUSSION

A. Performance Analysis

For first query PostgreSQL performed about 3x faster than MongoDB and about 13x faster than Neo4j. This is may be because:

TABLE III

Query	Metric	PostgreSQL	MongoDB	Neo4j
Q1	avg (s)	7.72	26.2	101.77
Q2	avg (s)	0.02	0.03	2.23
Q3	avg (s)	3.39	16.19	2.29
Q1	std dev (s)	0.25	0.37	1.41
Q2	std dev (s)	0.002	0.003	0.35
Q3	std dev (s)	0.17	0.16	0.21

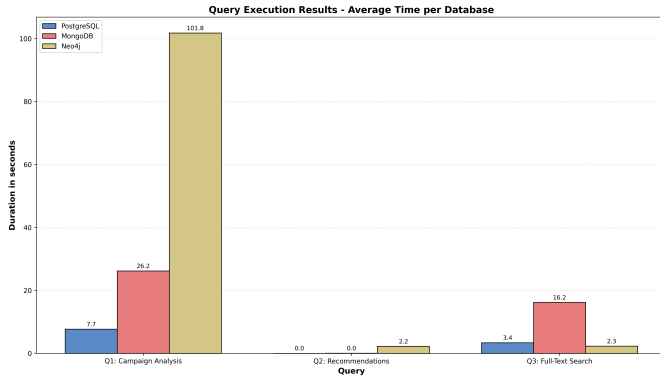


Fig. 4. Query execution times

- PostgreSQL's Hash Aggregate is optimized for GROUP BY operations.
- Indexes on campaign_id and user_id enable fast lookups
- MongoDB's aggregation pipeline creates intermediate documents at each stage
- Neo4j's relationship traversal overhead for 3 million messages

For second query PostgreSQL performed 1.3x faster than MongoDB and 86 x faster than Neo4j. This may be because:

- Simple aggregations with CASE statements execute very fast in SQL.
- Foreign key relationships enable efficient JOINS.
- Query planner optimizes the entire query as one execution plan
- MongoDB's 'lookup' operations are slower than SQL

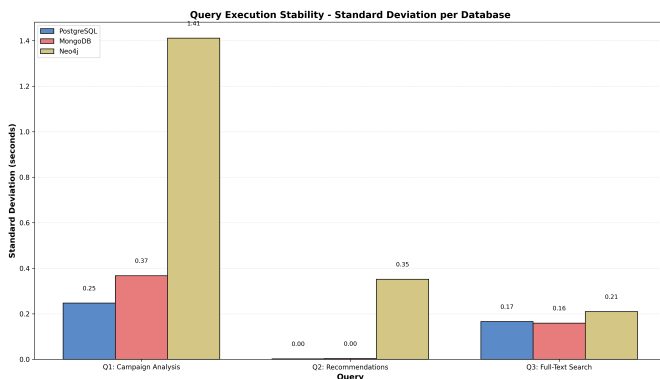


Fig. 5. Standard deviation of time in runs

JOINS

- Neo4j's OPTIONAL MATCH and list comprehension add significant overhead

For last third query Neo4j performed 1.5x faster than PostgreSQL and 7x faster than MongoDB. This may be because:

- Graph traversal is native to Neo4j (PRODUCT_BELONG_TO_CATEGORY relationships).
- No JOIN overhead - relationships are pre-computed direct pointers.
- Cypher pattern matching is optimized for graph operations.
- PostgreSQL requires ILIKE operations on text fields.
- MongoDB's 'split' and 'unwind' operations are expensive.

B. Best model for this data

After performing these tests i suggest a better approach to store this data. I recommend on using hybrid model - this model will use all 3 databases, each of them will contain portion of data, which is best suited for this database. I think this data distribution will be the best:

- PostgreSQL: Users, Products, Categories, Campaigns (reference data)
- MongoDB: Events, Messages (high-volume operational data)
- Neo4j: Friends, Social relationships (graph traversals)

To combine these databases i suggest to use a distributed cache and message broker database such as **Redis** or similar to it. All of these databases will be connected via some API that will direct queries to optimal database. This model is more complex, expensive and you need to use multiple query languages. But for really big data storage i think this model will be the best. I do not know how to exactly make this model, but i can create needed tables, collection and nodes. In the project repository i made the sql script to create PostgreSQL part and python script to load data in. The schema of this model is listed below in figure 6:

HYBRID DATA MODEL

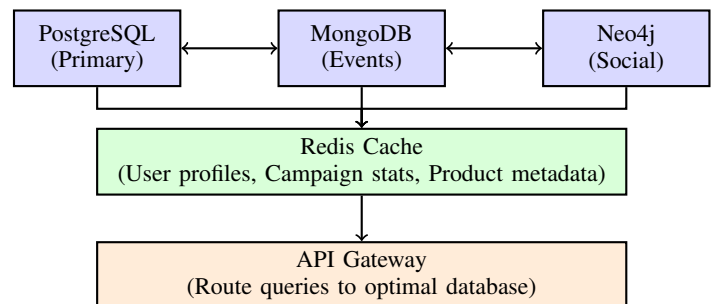


Fig. 6. Hybrid Data Model Architecture

V. CONCLUSION

The results indicate there is no database model that achieves all tasks faster than the others. PostgreSQL performs the best when the task involves analytical aggregations and complex JOINS (Q1 and Q2). Neo4j works faster for queries involving graph traversals and relationship patterns (Q3). MongoDB performs efficiently for simple lookups and operational workloads with embedded documents (Q2). My results of experiment confirm that important factors affecting database performance are: the domain of the application, data model design choices, and query types. For analytical workloads with complex aggregations, i think PostgreSQL is the best. For social network analysis with relationship traversals, Neo4j is the best due to graph structure of the data. For operational workloads with flexible schema requirements, MongoDB is very good and easy to work with.

For real usage with really big amount of data,i suggest to use a hybrid approach that leverages the strengths of each database type while mitigating their weaknesses.